

Compact Diffusion Models:

A Comprehensive Study of Knowledge Distillation, Pruning, Quantization, and Symmetry-Aware Architectures

Ajit Kumar
Pronay Dutta

December 27, 2025

Abstract

Diffusion models have emerged as state-of-the-art generative models, achieving unprecedented success in high-fidelity image synthesis by learning to reverse a stochastic noise corruption process. However, their deployment on resource-constrained platforms remains challenging due to substantial computational and memory requirements. This comprehensive report presents a rigorous mathematical framework for developing compact diffusion models through the strategic integration of knowledge distillation, structured pruning, quantization, and symmetry-aware architectural design.

We provide detailed theoretical foundations for each compression technique, including complete derivations of training objectives, convergence properties, and performance bounds. Our experimental implementation on CIFAR-10, while limited by computational constraints to 10,000 images and approximately 200 training epochs, demonstrates the practical feasibility of achieving significant model compression (74% size reduction) with controlled performance degradation. Crucially, we introduce a novel theoretical framework for incorporating physical symmetries and invariances into diffusion model architectures, drawing from principles in geometric deep learning, equivariant neural networks, and physics-informed machine learning. This symmetry-aware perspective not only enhances data efficiency and model compactness but also ensures that generated samples respect fundamental conservation laws and geometric constraints relevant to specific application domains.

The report concludes with a thorough analysis of current limitations, including the impact of aggressive compression on Frchet Inception Distance (FID) scores, and outlines concrete directions for future research in latent diffusion models, advanced quantization schemes, and symmetry-preserving sampling algorithms.

Contents

1	Introduction	1
1.1	Motivation and Context	1
1.2	Compression Paradigms for Neural Networks	1
1.3	Symmetry and Inductive Bias	2
1.4	Objectives and Contributions	2
1.5	Document Organization	3
2	Mathematical Foundations of Diffusion Models	4
2.1	Probabilistic Framework	4
2.1.1	Forward Diffusion Process	4
2.1.2	Reverse Diffusion Process	6
2.1.3	Training Objective: Variational Lower Bound	7
2.1.4	Noise Prediction Parameterization	9
2.2	Score-Based Perspective	9
2.2.1	Score Function and Langevin Dynamics	9
2.2.2	Denoising Score Matching	9
2.2.3	Connection to Diffusion Models	10
2.3	Stochastic Differential Equation Formulation	10
2.3.1	Forward SDE	10
2.3.2	Reverse SDE	10
2.3.3	Probability Flow ODE	11
2.4	Classifier-Free Guidance	11
3	Model Compression Techniques: Theory and Practice	12
3.1	Knowledge Distillation	12
3.1.1	Classical Distillation Framework	12
3.1.2	Distillation for Diffusion Models	12
3.1.3	Progressive Distillation for Diffusion	13
3.1.4	Consistency Distillation	14
3.2	Structured Pruning	14
3.2.1	Pruning Formulations	14
3.2.2	Importance Criteria	15
3.2.3	Iterative Pruning with Fine-Tuning	15
3.2.4	Dependency Graphs for Structured Pruning	15
3.2.5	Layer-Wise Sensitivity Analysis	16
3.2.6	Pruning for Diffusion Models	16
3.3	Quantization	17
3.3.1	Uniform Quantization	17

3.3.2	Quantization-Aware Training (QAT)	17
3.3.3	Per-Channel vs Per-Tensor Quantization	18
3.3.4	Mixed-Precision Quantization	18
3.3.5	Quantization for Diffusion Models	18
4	Efficient Training Techniques	19
4.1	Mixed Precision Training	19
4.1.1	Floating-Point Representations	19
4.1.2	Mixed Precision Training Algorithm	20
4.1.3	Loss Scaling	20
4.1.4	Selective Precision	21
4.2	Exponential Moving Average (EMA)	21
4.2.1	EMA Update Rule	21
4.2.2	Bias Correction	21
4.2.3	EMA for Batch Normalization	22
4.2.4	EMA Benefits for Diffusion Models	22
4.3	Gradient Accumulation	22
4.4	Optimizer Selection	22
4.4.1	AdamW	22
4.4.2	Learning Rate Schedules	23
5	Model Architecture	24
5.1	U-Net Architecture Overview	24
5.1.1	Encoder-Decoder Structure	24
5.2	Building Blocks	24
5.2.1	Double Convolution Block	24
5.2.2	Self-Attention Layer	25
5.2.3	Downsampling and Upsampling	26
5.3	Conditioning Mechanisms	27
5.3.1	Timestep Embedding	27
5.3.2	Class Conditioning	27
5.3.3	Adaptive Group Normalization (AdaGN)	28
5.4	Compact Design Choices	28
5.4.1	Channel Compression Factor	28
5.4.2	Attention Placement	28
5.4.3	Reduced Attention Heads	28
5.4.4	Depthwise Separable Convolutions	28
6	Symmetry-Aware Diffusion Models	29
6.1	Motivation and Mathematical Foundations	29
6.1.1	Symmetries in Physical Systems	29
6.1.2	Group Theory Fundamentals	29
6.1.3	Equivariance and Invariance	30
6.1.4	Relevant Symmetry Groups	30
6.2	Equivariant Neural Networks	31
6.2.1	Group Convolution	31
6.2.2	Steerable Convolutions	31
6.2.3	$E(3)$ -Equivariant Graph Networks	31
6.3	Symmetry-Aware Diffusion Models	32

6.3.1	Equivariant Forward Process	32
6.3.2	Equivariant Denoising Network	32
6.3.3	Score Function Equivariance	32
6.3.4	Gauge Equivariance	32
6.4	Incorporating Conservation Laws	33
6.4.1	Continuous Symmetries and Noether’s Theorem	33
6.4.2	Constrained Diffusion	33
6.4.3	Energy-Based Models and Hamiltonian Dynamics	34
6.5	Symmetry-Induced Model Compression	34
6.5.1	Parameter Sharing via Equivariance	34
6.5.2	Factorized Generation	34
6.6	Examples and Applications	35
6.6.1	Image Generation with Rotation Equivariance	35
6.6.2	Molecular Conformation Generation	35
6.6.3	Point Cloud Generation	35
6.6.4	Physics Simulation	35
6.7	Combining Symmetry with Other Compression Techniques	36
6.7.1	Symmetry-Aware Pruning	36
6.7.2	Symmetry-Aware Quantization	36
6.7.3	Distillation with Symmetry Constraints	36
7	Experimental Results and Analysis	37
7.1	Experimental Setup	37
7.1.1	Dataset	37
7.1.2	Model Configuration	37
7.1.3	Training Details	38
7.1.4	Hardware	38
7.2	Quantitative Results	38
7.2.1	Model Size and Inference Time	38
7.2.2	Generation Quality: FID Scores	39
7.2.3	Training Curves	39
7.2.4	Generated Samples	40
7.3	Critical Analysis	42
7.3.1	Factors Contributing to High FID Scores	42
7.3.2	Comparison to Baselines	43
7.3.3	Distillation Effectiveness	43
7.3.4	Pruning Trade-offs	44
7.4	Ablation Studies	44
7.4.1	Impact of Distillation Loss Components	44
7.4.2	Impact of EMA	44
7.5	Limitations	44
7.5.1	Computational Constraints	44
7.5.2	Architectural Choices	45
7.5.3	Evaluation Metrics	45

8	Conclusion and Future Directions	46
8.1	Summary of Contributions	46
8.2	Key Insights	46
8.3	Future Research Directions	47
8.3.1	Improved Training Protocols	47
8.3.2	Advanced Compression	47
8.3.3	Latent Diffusion Models	47
8.3.4	Symmetry-Aware Architectures	47
8.3.5	Theoretical Analysis	48
8.3.6	Application Domains	48
8.4	Broader Impact	48
8.4.1	Positive Impacts	48
8.4.2	Potential Concerns	48
8.5	Concluding Remarks	49
A	Additional Experimental Details	51
A.1	Hyperparameter Settings	51
A.2	Computational Requirements	51

List of Figures

2.1	Illustration of the forward diffusion process progressively adding Gaussian noise to data samples over T timesteps.	5
5.1	Complete U-Net architecture showing encoder, bottleneck, and decoder paths with skip connections.	25
5.2	ResNet-style convolutional block with skip connection.	26
5.3	Self-attention block with layer normalization and feed-forward network. . .	26
5.4	Upsampling block with bilinear interpolation and convolution.	27
7.1	MSE training loss for teacher model over 200 epochs. Loss decreases steadily but has not fully converged.	39
7.2	MSE training loss for student model with knowledge distillation. Combined loss shown; standard diffusion loss and distillation loss behave similarly. . .	40
7.3	Sample images generated by the teacher model after 200 epochs. Recognizable class features but lacking fine details and exhibiting noise artifacts. . .	40
7.4	Sample images generated by the distilled student model. Quality comparable to teacher but with slightly reduced sharpness.	41
7.5	Sample images generated by the pruned student model (30% sparsity). Visible quality degradation with increased noise and loss of fine details. . .	42

List of Tables

7.1	Model size and inference time comparison. Batch size = 8, $T = 1000$ sampling steps.	38
7.2	Generation quality metrics. Lower FID is better; higher Inception Score is better. For reference, fully-trained DDPM on full CIFAR-10 achieves FID ≈ 3 –5.	39
7.3	Comparison to published baselines. Note: direct comparison is unfair due to different training regimes.	43
7.4	Ablation study on distillation loss components. Feature matching provides small but consistent improvement.	44
7.5	Effect of using EMA weights for evaluation. EMA consistently improves FID by 8–10 points.	44
A.1	Complete hyperparameter settings for all experiments.	51
A.2	Computational requirements for model training and inference.	51

Chapter 1

Introduction

1.1 Motivation and Context

The field of generative modeling has undergone a paradigm shift with the advent of diffusion probabilistic models (??). These models learn to generate high-dimensional data by iteratively denoising samples from a simple prior distribution, typically an isotropic Gaussian. Unlike generative adversarial networks (GANs) (?) or variational autoencoders (VAEs) (?), diffusion models offer stable training dynamics, mode coverage guarantees, and interpretable generation processes grounded in stochastic differential equations (SDEs) and thermodynamic principles.

Despite these advantages, diffusion models present significant computational challenges:

1. **Iterative Generation:** Sampling requires sequential evaluation through hundreds or thousands of diffusion timesteps, leading to prohibitive inference latency.
2. **Model Scale:** State-of-the-art diffusion models often contain hundreds of millions of parameters, demanding substantial memory resources.
3. **Training Complexity:** Learning the score function across the entire diffusion trajectory necessitates extensive computational resources and training time.

These constraints severely limit deployment scenarios, particularly for edge devices, mobile platforms, and real-time applications. Addressing this challenge requires a multi-faceted approach combining model compression, architectural innovations, and algorithm-level optimizations.

1.2 Compression Paradigms for Neural Networks

Model compression techniques have been extensively studied in the context of discriminative models (???). These methods broadly fall into four categories:

Knowledge Distillation Transferring learned representations from a large teacher network to a compact student network by minimizing divergence between their output distributions (?).

Pruning Systematically removing redundant or less important parameters based on magnitude, gradient, or higher-order sensitivity metrics (??).

Quantization Reducing numerical precision of weights and activations from 32-bit floating-point to lower bit-widths (e.g., 8-bit integers) (??).

Architectural Search Automatically discovering efficient network topologies optimized for specific compute/accuracy trade-offs (?).

While these techniques have proven effective for convolutional and transformer-based discriminative models, their application to generative diffusion models introduces unique challenges related to maintaining sample quality, preserving mode coverage, and ensuring stable multi-step generation processes.

1.3 Symmetry and Inductive Bias

A parallel development in machine learning has been the recognition that incorporating domain-specific symmetries and invariances as inductive biases can dramatically improve sample efficiency, generalization, and model compactness (??). In physics-informed machine learning, respecting fundamental symmetries (e.g., Galilean invariance, rotational equivariance, gauge symmetry) ensures that learned representations align with physical laws (?).

For diffusion models, symmetry-aware architectures offer several advantages:

- **Parameter Efficiency:** Weight sharing induced by group equivariance reduces effective parameter count.
- **Data Efficiency:** Models that respect symmetries require fewer training examples to generalize.
- **Physical Consistency:** Generated samples automatically satisfy conservation laws and geometric constraints.
- **Compositional Generation:** Symmetries enable factorization of generation process across independent modes.

1.4 Objectives and Contributions

The primary objectives of this work are:

1. **Theoretical Framework:** Establish rigorous mathematical foundations for each compression technique in the context of diffusion models, including detailed derivations of training objectives and performance bounds.
2. **Empirical Validation:** Demonstrate practical feasibility through implementation and evaluation on CIFAR-10, analyzing trade-offs between compression ratio, inference speed, and generation quality.
3. **Symmetry Integration:** Develop a comprehensive framework for incorporating physical symmetries into diffusion architectures, with concrete proposals for equivariant denoising networks.

4. **Analysis and Future Directions:** Critically evaluate current limitations and propose concrete research directions for advancing compact diffusion models.

Our contributions include:

- Complete mathematical derivations for diffusion model training objectives, including variational lower bounds and score matching formulations.
- Detailed analysis of knowledge distillation for diffusion models, with novel loss formulations incorporating intermediate feature matching and temporal consistency.
- Rigorous treatment of structured pruning with dependency graph analysis and layer-wise sensitivity metrics.
- Comprehensive coverage of quantization-aware training with gradient approximation techniques.
- Novel theoretical framework for symmetry-aware diffusion models, connecting concepts from geometric deep learning to generative modeling.
- Experimental results demonstrating 74% model size reduction with controlled performance degradation.

1.5 Document Organization

This report is organized as follows:

Chapter 2 provides comprehensive mathematical background on diffusion probabilistic models, including forward/reverse processes, training objectives, and connections to score-based generative modeling and SDEs.

Chapter 3 presents detailed theoretical foundations for model compression techniques: knowledge distillation, pruning, and quantization, with complete derivations and convergence analyses.

Chapter 4 discusses practical training techniques including mixed precision training, exponential moving averages, and gradient scaling strategies.

Chapter 5 describes the conditional U-Net architecture employed in our implementation, with emphasis on design choices for compact models.

Chapter 6 introduces a comprehensive framework for symmetry-aware diffusion models, covering group theory fundamentals, equivariant architectures, and physics-informed constraints.

Chapter 7 presents experimental methodology, results, and critical analysis of our compact diffusion model implementation.

Chapter 8 concludes with discussion of limitations and future research directions.

Chapter 2

Mathematical Foundations of Diffusion Models

This chapter establishes the rigorous mathematical framework underlying diffusion probabilistic models. We begin with the fundamental forward and reverse diffusion processes, derive the variational lower bound training objective, and explore connections to score-based generative modeling and stochastic differential equations.

2.1 Probabilistic Framework

2.1.1 Forward Diffusion Process

Let $\mathbf{x}_0 \sim q_{\text{data}}(\mathbf{x}_0)$ denote a sample from the true data distribution. The forward diffusion process is a discrete-time Markov chain that gradually corrupts $\mathbf{x}_0$ by adding Gaussian noise over T timesteps:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}), \quad (2.1)$$

where $\{\beta_t\}_{t=1}^T$ is a variance schedule satisfying $0 < \beta_t < 1$ for all t .

Reparameterization and Closed Form

Using the reparameterization trick, we can express Equation (2.1) as:

$$\mathbf{x}_t = \sqrt{1 - \beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_{t-1}, \quad \boldsymbol{\epsilon}_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (2.2)$$

A key property of Gaussian diffusion processes is that the marginal distribution $q(\mathbf{x}_t|\mathbf{x}_0)$ can be computed in closed form. Define:

$$\alpha_t = 1 - \beta_t, \quad (2.3)$$

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s. \quad (2.4)$$

Then by the Markov property and Gaussian closure:

Proposition 2.1.1 (Closed-Form Forward Process). *For any timestep $t \in \{1, \dots, T\}$:*

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}). \quad (2.5)$$

Figure 2.1: Illustration of the forward diffusion process progressively adding Gaussian noise to data samples over T timesteps.

Proof. We proceed by induction. The base case $t = 1$ follows immediately from Equation (2.1) with $\bar{\alpha}_1 = \alpha_1 = 1 - \beta_1$.

For the inductive step, assume the result holds for $t - 1$. Then:

$$\begin{aligned} q(\mathbf{x}_t|\mathbf{x}_0) &= \int q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1}|\mathbf{x}_0) d\mathbf{x}_{t-1} \\ &= \int \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})\mathcal{N}(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0, (1 - \bar{\alpha}_{t-1})\mathbf{I}) d\mathbf{x}_{t-1}. \end{aligned}$$

This is a convolution of two Gaussians. Using the identity for Gaussian convolution:

$$\mathcal{N}(\mathbf{y}; A\mathbf{x}, \Sigma_1) * \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma_2) = \mathcal{N}(\mathbf{y}; A\boldsymbol{\mu}, A\Sigma_2A^\top + \Sigma_1),$$

we obtain:

$$\begin{aligned} q(\mathbf{x}_t|\mathbf{x}_0) &= \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0, \alpha_t(1 - \bar{\alpha}_{t-1})\mathbf{I} + \beta_t\mathbf{I}) \\ &= \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t\bar{\alpha}_{t-1}}\mathbf{x}_0, (\alpha_t - \alpha_t\bar{\alpha}_{t-1} + \beta_t)\mathbf{I}) \\ &= \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}), \end{aligned}$$

where we used $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$ and $\alpha_t - \alpha_t \bar{\alpha}_{t-1} + \beta_t = \alpha_t(1 - \bar{\alpha}_{t-1}) + (1 - \alpha_t) = 1 - \alpha_t \bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t$. $\square$

Direct Sampling via Reparameterization

Equation (2.5) enables direct sampling of $\mathbf{x}_t$ given $\mathbf{x}_0$ without iterating through intermediate timesteps:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (2.6)$$

This formulation explicitly separates the signal contribution $\sqrt{\bar{\alpha}_t} \mathbf{x}_0$ from the noise contribution $\sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$.

Variance Schedule Design

The choice of variance schedule $\{\beta_t\}_{t=1}^T$ significantly impacts model performance. Common schedules include:

Linear Schedule (?):

$$\beta_t = \beta_{\min} + \frac{t-1}{T-1}(\beta_{\max} - \beta_{\min}), \quad (2.7)$$

typically with $\beta_{\min} = 10^{-4}$ and $\beta_{\max} = 0.02$.

Cosine Schedule (?):

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2 \left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2} \right), \quad (2.8)$$

where s is a small offset (e.g., $s = 0.008$).

Learned Schedule : Treating $\{\beta_t\}$ or $\{\bar{\alpha}_t\}$ as learnable parameters optimized jointly with model weights.

2.1.2 Reverse Diffusion Process

The generative process involves reversing the forward diffusion. The true reverse conditional $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is intractable as it requires integrating over all possible paths. However, for sufficiently small β_t , the reverse process is also Gaussian (??).

We approximate the reverse process using a parameterized model p_θ :

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)), \quad (2.9)$$

where $\boldsymbol{\mu}_\theta$ and Σ_θ are neural network outputs. For simplicity, the covariance is often fixed to $\Sigma_\theta(\mathbf{x}_t, t) = \sigma_t^2 \mathbf{I}$, where σ_t is chosen according to:

$$\sigma_t^2 = \beta_t \quad \text{or} \quad \sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t. \quad (2.10)$$

Posterior Distribution

A crucial insight is that the true reverse conditional can be computed analytically when conditioning on $\mathbf{x}_0$:

Lemma 2.1.2 (Reverse Posterior).

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}), \quad (2.11)$$

where

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t. \quad (2.12)$$

Proof. By Bayes' theorem:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}.$$

By the Markov property, $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1})$. Substituting the Gaussian forms:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \propto \exp \left(-\frac{1}{2} \left[\frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^2}{1 - \bar{\alpha}_t} \right] \right).$$

Expanding and completing the square in $\mathbf{x}_{t-1}$:

$$\begin{aligned} & \frac{1}{\beta_t} (\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^2 + \frac{1}{1 - \bar{\alpha}_{t-1}} (\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)^2 \\ &= \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right) \mathbf{x}_{t-1}^2 - 2 \left(\frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right) \mathbf{x}_{t-1} + \text{const.} \end{aligned}$$

The coefficient of $\mathbf{x}_{t-1}^2$ gives the precision:

$$\tilde{\beta}_t^{-1} = \frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1 - \bar{\alpha}_{t-1})} = \frac{1 - \bar{\alpha}_t}{\beta_t(1 - \bar{\alpha}_{t-1})},$$

where we used $\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t = 1 - \bar{\alpha}_t$.

The mean follows from:

$$\tilde{\boldsymbol{\mu}}_t = \tilde{\beta}_t \left(\frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0 \right).$$

Substituting $\tilde{\beta}_t = \frac{(1 - \bar{\alpha}_{t-1})\beta_t}{1 - \bar{\alpha}_t}$ yields Equation (2.12). $\square$

2.1.3 Training Objective: Variational Lower Bound

The training objective is derived from maximizing the log-likelihood $\log p_\theta(\mathbf{x}_0)$ of the data. We use the variational lower bound (VLB):

Theorem 2.1.3 (Variational Lower Bound for Diffusion Models).

$$\log p_\theta(\mathbf{x}_0) \geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = -\mathcal{L}_{VLB}, \quad (2.13)$$

where

$$\mathcal{L}_{VLB} = \mathcal{L}_T + \sum_{t=2}^T \mathcal{L}_{t-1} + \mathcal{L}_0, \quad (2.14)$$

with

$$\mathcal{L}_T = KL(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T)), \quad (2.15)$$

$$\mathcal{L}_{t-1} = KL(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)), \quad (2.16)$$

$$\mathcal{L}_0 = -\log p_\theta(\mathbf{x}_0|\mathbf{x}_1). \quad (2.17)$$

Proof. Starting from the log-likelihood and introducing the forward process q :

$$\begin{aligned} \log p_\theta(\mathbf{x}_0) &= \log \int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T} \\ &= \log \int \frac{p_\theta(\mathbf{x}_{0:T})q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} d\mathbf{x}_{1:T} \\ &= \log \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] \\ &\geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] \quad (\text{Jensen's inequality}). \end{aligned}$$

Expanding the joint distributions:

$$\begin{aligned} p_\theta(\mathbf{x}_{0:T}) &= p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t), \\ q(\mathbf{x}_{1:T}|\mathbf{x}_0) &= \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}). \end{aligned}$$

Therefore:

$$\mathbb{E}_q \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = \mathbb{E}_q \left[\log p(\mathbf{x}_T) + \sum_{t=1}^T \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) - \sum_{t=1}^T \log q(\mathbf{x}_t|\mathbf{x}_{t-1}) \right].$$

Using $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)$ and telescoping:

$$\begin{aligned} \sum_{t=1}^T \log q(\mathbf{x}_t|\mathbf{x}_{t-1}) &= \log q(\mathbf{x}_T|\mathbf{x}_0) + \sum_{t=1}^{T-1} \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t+1}|\mathbf{x}_0)} \\ &= \log q(\mathbf{x}_T|\mathbf{x}_0) + \sum_{t=2}^T \log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) - \log q(\mathbf{x}_1|\mathbf{x}_0). \end{aligned}$$

Collecting terms yields Equations (2.15)–(2.17). $\square$

Simplified Training Objective

The term $\mathcal{L}_T$ is constant (no trainable parameters), and $\mathcal{L}_0$ can be modeled with a separate decoder. The key training signal comes from $\mathcal{L}_{t-1}$. Since both q and p_θ are Gaussians with fixed variance, the KL divergence simplifies to:

$$\mathcal{L}_{t-1} = \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2 \right]. \quad (2.18)$$

2.1.4 Noise Prediction Parameterization

Rather than directly predicting $\boldsymbol{\mu}_\theta$, modern implementations predict the noise $\boldsymbol{\epsilon}$ added during the forward process (?). Using Equation (2.6):

$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}). \quad (2.19)$$

Substituting into Equation (2.12):

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon} \right). \quad (2.20)$$

Thus, we parameterize:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right), \quad (2.21)$$

where $\boldsymbol{\epsilon}_\theta : \mathbb{R}^d \times \{1, \dots, T\} \rightarrow \mathbb{R}^d$ is a neural network predicting the noise.

The simplified training objective becomes:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t \sim \mathcal{U}(1, T), \mathbf{x}_0, \boldsymbol{\epsilon}} [\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2], \quad (2.22)$$

where $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}$ and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Remark 2.1.4. Ho et al. (?) found empirically that dropping the weighting factor $\frac{1}{2\sigma_t^2}$ improves sample quality, though this deviates from the strict VLB objective.

2.2 Score-Based Perspective

An alternative but equivalent formulation views diffusion models as learning the score function $\nabla_{\mathbf{x}} \log q(\mathbf{x})$ (??).

2.2.1 Score Function and Langevin Dynamics

The score function measures the gradient of the log-density. Given a smooth density $p(\mathbf{x})$, samples can be generated via Langevin dynamics:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\eta}{2} \nabla_{\mathbf{x}} \log p(\mathbf{x}_k) + \sqrt{\eta} \mathbf{z}_k, \quad \mathbf{z}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (2.23)$$

which converges to samples from $p(\mathbf{x})$ as $\eta \rightarrow 0$ and $k \rightarrow \infty$.

2.2.2 Denoising Score Matching

Direct score matching is challenging due to the need to compute $\nabla_{\mathbf{x}} \log q(\mathbf{x})$. Vincent (?) showed that the score can be learned by training a model to denoise corrupted data:

Theorem 2.2.1 (Denoising Score Matching). *Let $q_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}}; \mathbf{x}, \sigma^2 \mathbf{I})$ be a noise distribution. Then:*

$$\nabla_{\mathbf{x}} \log q_\sigma(\mathbf{x}) = \mathbb{E}_{q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})} [\nabla_{\mathbf{x}} \log q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})] = \mathbb{E}_{\boldsymbol{\epsilon}} \left[-\frac{\boldsymbol{\epsilon}}{\sigma} \right] = -\frac{1}{\sigma^2} \mathbb{E}_{q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})} [\tilde{\mathbf{x}} - \mathbf{x}]. \quad (2.24)$$

Thus, training a model $\mathbf{s}_\theta(\mathbf{x}, \sigma)$ to predict $\frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2}$ is equivalent to estimating the score.

2.2.3 Connection to Diffusion Models

For diffusion models, the score of the forward marginal is:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0) = -\frac{1}{1 - \bar{\alpha}_t}(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0) = -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}. \quad (2.25)$$

Therefore, predicting the noise $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is equivalent to predicting the score (up to scaling):

$$\mathbf{s}_\theta(\mathbf{x}_t, t) = -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t). \quad (2.26)$$

2.3 Stochastic Differential Equation Formulation

Song et al. (?) unified diffusion models and score-based models through the lens of stochastic differential equations.

2.3.1 Forward SDE

The continuous-time limit of the forward diffusion process is described by the SDE:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t) dt + g(t) d\mathbf{w}, \quad (2.27)$$

where $\mathbf{w}$ is a standard Wiener process, $\mathbf{f}(\mathbf{x}, t)$ is the drift coefficient, and $g(t)$ is the diffusion coefficient.

For the variance-preserving (VP) formulation:

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}. \quad (2.28)$$

For the variance-exploding (VE) formulation:

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w}. \quad (2.29)$$

2.3.2 Reverse SDE

Anderson (?) showed that the reverse-time SDE is:

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{w}}, \quad (2.30)$$

where $\bar{\mathbf{w}}$ is a reverse-time Wiener process and p_t is the marginal density at time t .

Substituting the score approximation $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \approx \mathbf{s}_\theta(\mathbf{x}, t)$:

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t) \mathbf{s}_\theta(\mathbf{x}, t)] dt + g(t) d\bar{\mathbf{w}}. \quad (2.31)$$

This SDE can be integrated numerically using methods like Euler-Maruyama or higher-order solvers (?).

2.3.3 Probability Flow ODE

Song et al. (?) derived a deterministic ODE that generates the same marginal densities:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\nabla_{\mathbf{x}}\log p_t(\mathbf{x}). \quad (2.32)$$

This formulation enables:

- Exact likelihood computation via change of variables.
- Latent space manipulation and interpolation.
- Connection to neural ODEs and normalizing flows.

2.4 Classifier-Free Guidance

For conditional generation $p(\mathbf{x}|y)$ where y is a class label or text prompt, classifier-free guidance (?) modifies the score:

$$\tilde{\epsilon}_{\theta}(\mathbf{x}_t, t, y) = \epsilon_{\theta}(\mathbf{x}_t, t, \emptyset) + s \cdot (\epsilon_{\theta}(\mathbf{x}_t, t, y) - \epsilon_{\theta}(\mathbf{x}_t, t, \emptyset)), \quad (2.33)$$

where $s \geq 1$ is the guidance scale and $\emptyset$ denotes the unconditional (null) class. This corresponds to sampling from:

$$\tilde{p}(\mathbf{x}_t) \propto p(\mathbf{x}_t)^{1-s} p(\mathbf{x}_t|y)^s. \quad (2.34)$$

Higher guidance scales increase sample fidelity to the condition but may reduce diversity.

Chapter 3

Model Compression Techniques: Theory and Practice

This chapter presents comprehensive theoretical foundations for three primary model compression techniques applicable to diffusion models: knowledge distillation, structured pruning, and quantization. For each method, we provide complete mathematical derivations, convergence analyses, and practical considerations.

3.1 Knowledge Distillation

Knowledge distillation (?) transfers knowledge from a large teacher model to a compact student model by training the student to mimic the teacher’s behavior.

3.1.1 Classical Distillation Framework

Temperature-Scaled Softmax

For classification tasks, let $\mathbf{z}_T$ and $\mathbf{z}_S$ denote teacher and student logits. The temperature-scaled softmax is:

$$p_i^{(T)}(\mathbf{z}_T, \tau) = \frac{\exp(z_{T,i}/\tau)}{\sum_j \exp(z_{T,j}/\tau)}, \quad (3.1)$$

where $\tau > 0$ is the temperature. As $\tau \rightarrow 0$, the distribution becomes peaked (one-hot), while $\tau \rightarrow \infty$ yields a uniform distribution.

Distillation Loss

The total loss combines task loss and distillation loss:

$$\mathcal{L}_{\text{distill}} = \alpha \mathcal{L}_{\text{task}}(\mathbf{z}_S, y) + \beta \tau^2 \text{KL}(p^{(T)}(\mathbf{z}_T, \tau) \| p^{(S)}(\mathbf{z}_S, \tau)), \quad (3.2)$$

where α, β are weighting coefficients and the τ^2 factor accounts for gradient scaling.

3.1.2 Distillation for Diffusion Models

For diffusion models, distillation operates on the noise prediction function $\epsilon_\theta(\mathbf{x}_t, t)$.

Direct Output Matching

The simplest approach matches student predictions to teacher predictions:

$$\mathcal{L}_{\text{distill}}^{\text{output}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\|\epsilon_T(\mathbf{x}_t, t) - \epsilon_S(\mathbf{x}_t, t)\|^2], \quad (3.3)$$

where subscripts T and S denote teacher and student.

Combined Objective

The student is trained with a weighted combination:

$$\mathcal{L}_{\text{total}} = \alpha \mathcal{L}_{\text{diffusion}} + \beta \mathcal{L}_{\text{distill}}^{\text{output}}, \quad (3.4)$$

where $\mathcal{L}_{\text{diffusion}}$ is the standard diffusion objective (Equation (2.22)).

Intermediate Feature Matching

Beyond final outputs, matching intermediate representations can improve distillation (?). Let $\mathbf{F}_T^{(\ell)}$ and $\mathbf{F}_S^{(\ell)}$ denote activations from layer ℓ of teacher and student. The feature matching loss is:

$$\mathcal{L}_{\text{feature}}^{(\ell)} = \mathbb{E} \left[\left\| \mathbf{F}_T^{(\ell)}(\mathbf{x}_t, t) - \Phi^{(\ell)}(\mathbf{F}_S^{(\ell)}(\mathbf{x}_t, t)) \right\|^2 \right], \quad (3.5)$$

where $\Phi^{(\ell)}$ is an optional projection layer to align dimensions.

The total distillation loss becomes:

$$\mathcal{L}_{\text{distill}} = \mathcal{L}_{\text{distill}}^{\text{output}} + \sum_{\ell \in \mathcal{L}} \gamma_{\ell} \mathcal{L}_{\text{feature}}^{(\ell)}, \quad (3.6)$$

where $\mathcal{L}$ is the set of layers for feature matching and γ_{ℓ} are layer-specific weights.

3.1.3 Progressive Distillation for Diffusion

Salimans and Ho (?) proposed progressive distillation specifically for accelerating diffusion model sampling.

Two-Step Distillation

The key idea is to train the student to predict in one step what the teacher predicts in two steps. Let $\mathbf{x}_t^{(0)} = \mathbf{x}_t$ and define:

$$\mathbf{x}_t^{(1)} = \sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0^{(0)} + \sqrt{1 - \bar{\alpha}_{t-1}} \epsilon_T(\mathbf{x}_t^{(0)}, t), \quad (3.7)$$

where $\hat{\mathbf{x}}_0^{(0)} = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t^{(0)} - \sqrt{1 - \bar{\alpha}_t} \epsilon_T(\mathbf{x}_t^{(0)}, t))$.

Then compute:

$$\mathbf{x}_t^{(2)} = \sqrt{\bar{\alpha}_{t-2}} \hat{\mathbf{x}}_0^{(1)} + \sqrt{1 - \bar{\alpha}_{t-2}} \epsilon_T(\mathbf{x}_t^{(1)}, t-1). \quad (3.8)$$

The student is trained to predict directly from $\mathbf{x}_t^{(0)}$ to $\mathbf{x}_t^{(2)}$:

$$\mathcal{L}_{\text{progressive}} = \mathbb{E} \left[\left\| \mathbf{x}_t^{(2)} - \left(\sqrt{\bar{\alpha}_{t-2}} \hat{\mathbf{x}}_0^{(S)} + \sqrt{1 - \bar{\alpha}_{t-2}} \epsilon_S(\mathbf{x}_t, t) \right) \right\|^2 \right]. \quad (3.9)$$

Iterative Halving

By repeatedly applying this procedure, the number of sampling steps can be reduced exponentially: $T \rightarrow T/2 \rightarrow T/4 \rightarrow \dots \rightarrow 1$.

3.1.4 Consistency Distillation

Song et al. (?) introduced consistency models that map any point on a diffusion trajectory to the origin:

$$\mathbf{f}_\theta(\mathbf{x}_t, t) \approx \mathbf{x}_0. \quad (3.10)$$

The consistency loss enforces:

$$\mathcal{L}_{\text{consistency}} = \mathbb{E} [\|\mathbf{f}_\theta(\mathbf{x}_{t+\Delta t}, t + \Delta t) - \mathbf{f}_\theta(\mathbf{x}_t, t)\|^2], \quad (3.11)$$

where $\mathbf{x}_{t+\Delta t}$ is obtained by one step of the diffusion ODE starting from $\mathbf{x}_t$.

This enables single-step generation: $\mathbf{x}_0 \approx \mathbf{f}_\theta(\mathbf{x}_T, T)$ where $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

3.2 Structured Pruning

Pruning reduces model complexity by removing parameters based on importance criteria (??).

3.2.1 Pruning Formulations

Unstructured Pruning

Unstructured pruning removes individual weights:

$$w_{ij} \leftarrow w_{ij} \cdot m_{ij}, \quad m_{ij} \in \{0, 1\}, \quad (3.12)$$

where m_{ij} is a binary mask. The mask is typically determined by magnitude thresholding:

$$m_{ij} = \mathbb{1}_{|w_{ij}| \geq \delta_p}, \quad (3.13)$$

where δ_p is chosen to achieve a target sparsity s (fraction of pruned weights).

Structured Pruning

Structured pruning removes entire channels, neurons, or blocks:

$$\mathbf{W}_{:,j} \leftarrow \mathbf{W}_{:,j} \cdot m_j, \quad m_j \in \{0, 1\}, \quad (3.14)$$

where $\mathbf{W}_{:,j}$ denotes the j -th output channel (column).

3.2.2 Importance Criteria

Magnitude-Based Pruning

The simplest criterion is weight magnitude:

$$\text{importance}(w_{ij}) = |w_{ij}|. \quad (3.15)$$

For structured pruning of channels:

$$\text{importance}(\mathbf{W}_{:,j}) = \|\mathbf{W}_{:,j}\|_1 \quad \text{or} \quad \|\mathbf{W}_{:,j}\|_2. \quad (3.16)$$

Gradient-Based Pruning

Gradient magnitude reflects sensitivity:

$$\text{importance}(w_{ij}) = \left| w_{ij} \cdot \frac{\partial \mathcal{L}}{\partial w_{ij}} \right|. \quad (3.17)$$

Second-Order Methods

The Optimal Brain Damage (OBD) criterion (?) uses second-order information:

$$\text{importance}(w_{ij}) = \frac{1}{2} h_{ij} w_{ij}^2, \quad (3.18)$$

where $h_{ij} = \frac{\partial^2 \mathcal{L}}{\partial w_{ij}^2}$ is the Hessian diagonal.

3.2.3 Iterative Pruning with Fine-Tuning

Aggressive one-shot pruning often degrades performance significantly. Iterative pruning alternates between pruning and fine-tuning:

Algorithm 1 Iterative Magnitude Pruning

- 1: **Input:** Trained model θ_0 , target sparsity s_{target} , iterations N
 - 2: **Output:** Pruned model θ_N
 - 3: **for** $i = 1$ to N **do**
 - 4: $s_i \leftarrow s_{\text{target}} \cdot (1 - (1 - i/N)^3)$ {Cubic schedule}
 - 5: Compute importance scores for all weights
 - 6: Set bottom s_i fraction of weights to zero
 - 7: Fine-tune for K steps to recover performance
 - 8: **end for**
 - 9: **return** θ_N
-

3.2.4 Dependency Graphs for Structured Pruning

When pruning entire channels, dependencies must be tracked:

- **Conv Layer:** Pruning output channel j removes $\mathbf{W}_{:,j,:}$ and requires removing corresponding input channel j in the next layer.

- **Skip Connections:** Residual connections constrain pruning both branches must maintain compatible dimensions.
- **Batch Normalization:** Pruning a channel requires removing corresponding BN parameters.

Formally, construct a dependency graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where:

- Vertices $\mathcal{V}$ represent channels/neurons.
- Edges $(\mathbf{v}_i, \mathbf{v}_j) \in \mathcal{E}$ indicate that pruning $\mathbf{v}_i$ requires pruning $\mathbf{v}_j$.

Pruning must respect the dependency graph to maintain architectural integrity.

3.2.5 Layer-Wise Sensitivity Analysis

Not all layers contribute equally to performance. We define layer sensitivity as:

Definition 3.2.1 (Layer Sensitivity). The sensitivity of layer ℓ to pruning is:

$$S^{(\ell)}(p) = \mathcal{L}_{\text{pruned}}^{(\ell)}(p) - \mathcal{L}_{\text{original}}, \quad (3.19)$$

where $\mathcal{L}_{\text{pruned}}^{(\ell)}(p)$ is the loss after pruning fraction p of layer ℓ 's parameters.

Layers with lower sensitivity can tolerate higher pruning rates. This leads to non-uniform pruning strategies:

$$p^{(\ell)} = p_{\text{base}} \cdot \exp(-\lambda S^{(\ell)}(p_{\text{probe}})), \quad (3.20)$$

where p_{base} is a baseline rate, λ controls sensitivity scaling, and p_{probe} is a small probe pruning fraction for estimating $S^{(\ell)}$.

3.2.6 Pruning for Diffusion Models

Diffusion models present unique challenges:

1. **Temporal Consistency:** Pruning must maintain consistency across timesteps.
2. **Multi-Resolution Features:** U-Net architectures have skip connections across resolutions.
3. **Attention Layers:** Self-attention is parameter-efficient but computationally expensive pruning attention heads requires careful analysis.

Timestep-Aware Pruning

Define per-timestep importance:

$$\text{importance}^{(t)}(\mathbf{W}_{:,j}) = \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\left\| \frac{\partial \mathcal{L}(\mathbf{x}_t, t)}{\partial \mathbf{W}_{:,j}} \right\|_2 \right], \quad (3.21)$$

and aggregate across critical timesteps:

$$\text{importance}(\mathbf{W}_{:,j}) = \sum_{t \in \mathcal{T}_{\text{critical}}} w_t \cdot \text{importance}^{(t)}(\mathbf{W}_{:,j}), \quad (3.22)$$

where $\mathcal{T}_{\text{critical}}$ contains timesteps contributing most to generation quality.

3.3 Quantization

Quantization reduces numerical precision of weights and activations (?).

3.3.1 Uniform Quantization

Quantization Mapping

Map continuous values to discrete levels:

$$w_q = \Delta \cdot \left\lfloor \frac{w - z}{\Delta} \right\rfloor + z, \quad (3.23)$$

where:

- $\Delta = \frac{w_{\max} - w_{\min}}{2^b - 1}$ is the quantization step (scale),
- b is the bit-width,
- z is the zero-point: $z = - \left\lfloor \frac{w_{\min}}{\Delta} \right\rfloor \Delta$.

Integer Representation

For efficient computation, represent as integers:

$$w_q = \Delta(q_{\text{int}} - z_{\text{int}}), \quad (3.24)$$

where $q_{\text{int}} \in \{0, 1, \dots, 2^b - 1\}$ and $z_{\text{int}} = \left\lfloor \frac{w_{\min}}{\Delta} \right\rfloor$.

3.3.2 Quantization-Aware Training (QAT)

Post-training quantization can cause significant accuracy drop. QAT simulates quantization during training.

Fake Quantization

Insert quantization operations in forward pass:

$$\tilde{w} = Q(w) = \Delta \cdot \text{clip} \left(\left\lfloor \frac{w}{\Delta} \right\rfloor, 0, 2^b - 1 \right), \quad (3.25)$$

where clip ensures values stay within representable range.

Straight-Through Estimator (STE)

The rounding operation is non-differentiable. The straight-through estimator (?) approximates gradients:

$$\frac{\partial Q(w)}{\partial w} \approx \begin{cases} 1, & \text{if } w_{\min} \leq w \leq w_{\max}, \\ 0, & \text{otherwise.} \end{cases} \quad (3.26)$$

This allows gradient backpropagation through quantization layers.

3.3.3 Per-Channel vs Per-Tensor Quantization

Per-Tensor Quantization

A single scale Δ for the entire tensor:

$$\Delta_{\text{tensor}} = \frac{\max(\mathbf{W}) - \min(\mathbf{W})}{2^b - 1}. \quad (3.27)$$

Simple but suboptimal if different channels have different dynamic ranges.

Per-Channel Quantization

Separate scale for each output channel:

$$\Delta_j = \frac{\max(\mathbf{W}_{:,j}) - \min(\mathbf{W}_{:,j})}{2^b - 1}. \quad (3.28)$$

More accurate representation at the cost of storing multiple scales.

3.3.4 Mixed-Precision Quantization

Different layers may tolerate different bit-widths. Define bit-width allocation $\mathbf{b} = (b_1, \dots, b_L)$ for L layers. The optimization problem is:

$$\begin{aligned} \mathbf{b} \quad & \sum_{\ell=1}^L \text{size}^{(\ell)}(b_\ell) \\ \text{subject to} \quad & \mathcal{L}_{\text{quant}}(\mathbf{b}) \leq \mathcal{L}_{\text{target}}, \\ & b_\ell \in \{2, 4, 8, 16, 32\}, \quad \forall \ell. \end{aligned} \quad (3.29)$$

This is typically solved via reinforcement learning or evolutionary algorithms (?).

3.3.5 Quantization for Diffusion Models

Activation Quantization Challenges

Activation distributions vary across timesteps in diffusion models. Using fixed quantization ranges can clip important values at certain timesteps.

Solution: Dynamic quantization with timestep-dependent ranges:

$$\Delta^{(t)} = f_\theta(t), \quad z^{(t)} = g_\theta(t), \quad (3.30)$$

where f_θ and g_θ are small learned networks predicting scale and zero-point.

Quantization-Aware Distillation

Combine quantization and distillation:

$$\mathcal{L} = \alpha \mathcal{L}_{\text{diffusion}} + \beta \mathcal{L}_{\text{distill}} + \gamma \mathcal{L}_{\text{quant}}, \quad (3.31)$$

where $\mathcal{L}_{\text{quant}}$ encourages quantization-friendly weights:

$$\mathcal{L}_{\text{quant}} = \mathbb{E} [\|w - Q(w)\|^2]. \quad (3.32)$$

Chapter 4

Efficient Training Techniques

Training large diffusion models requires careful optimization of computational resources. This chapter covers mixed precision training, exponential moving averages, and gradient scaling strategies.

4.1 Mixed Precision Training

Mixed precision training (?) combines FP16 and FP32 arithmetic to accelerate training while maintaining numerical stability.

4.1.1 Floating-Point Representations

IEEE 754 Formats

- **FP32 (Single Precision)**: 1 sign bit, 8 exponent bits, 23 mantissa bits.
 - Range: $\approx 10^{-38}$ to 10^{38}
 - Precision: ≈ 7 decimal digits
- **FP16 (Half Precision)**: 1 sign bit, 5 exponent bits, 10 mantissa bits.
 - Range: $\approx 10^{-8}$ to 65504
 - Precision: ≈ 3 decimal digits

4.1.2 Mixed Precision Training Algorithm

Algorithm 2 Mixed Precision Training with Loss Scaling

```
1: Input: Model  $\theta$  (FP32), loss scale  $s$ , data  $\mathcal{D}$ 
2: Output: Trained model  $\theta$ 
3: while not converged do
4:   Sample batch  $(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}$ 
5:   Cast model to FP16:  $\theta_{\text{FP16}} \leftarrow \text{cast}(\theta)$ 
6:   Forward pass in FP16:  $\hat{\mathbf{y}} = f_{\theta_{\text{FP16}}}(\mathbf{x})$ 
7:   Compute loss in FP32:  $\mathcal{L} = \text{loss}(\hat{\mathbf{y}}, \mathbf{y})$ 
8:   Scale loss:  $\mathcal{L}_{\text{scaled}} = s \cdot \mathcal{L}$ 
9:   Backward pass: compute gradients  $\mathbf{g}_{\text{FP16}} = \nabla_{\theta} \mathcal{L}_{\text{scaled}}$ 
10:  Unscale gradients:  $\mathbf{g} = \mathbf{g}_{\text{FP16}}/s$ 
11:  Check for overflow/underflow; adjust  $s$  if necessary
12:  Update FP32 master weights:  $\theta \leftarrow \theta - \eta \mathbf{g}$ 
13: end while
```

4.1.3 Loss Scaling

Static Loss Scaling

Use a fixed scale factor s (e.g., $s = 2^{10} = 1024$):

$$\mathcal{L}_{\text{scaled}} = s \cdot \mathcal{L}. \quad (4.1)$$

Gradients are computed with respect to $\mathcal{L}_{\text{scaled}}$ then divided by s :

$$\mathbf{g} = \frac{1}{s} \nabla_{\theta} \mathcal{L}_{\text{scaled}} = \nabla_{\theta} \mathcal{L}. \quad (4.2)$$

Dynamic Loss Scaling

Automatically adjust s based on gradient statistics:

Algorithm 3 Dynamic Loss Scaling

```
1: Initialize  $s = 2^{15}$ ,  $s_{\min} = 1$ ,  $s_{\max} = 2^{20}$ 
2:  $n_{\text{stable}} = 0$ ,  $n_{\text{threshold}} = 2000$ 
3: while training do
4:   if gradients contain NaN or Inf then
5:      $s \leftarrow \max(s/2, s_{\min})$ 
6:      $n_{\text{stable}} \leftarrow 0$ 
7:     Skip weight update
8:   else
9:      $n_{\text{stable}} \leftarrow n_{\text{stable}} + 1$ 
10:    if  $n_{\text{stable}} \geq n_{\text{threshold}}$  then
11:       $s \leftarrow \min(2s, s_{\max})$ 
12:       $n_{\text{stable}} \leftarrow 0$ 
13:    end if
14:  end if
15: end while
```

4.1.4 Selective Precision

Certain operations must remain in FP32 for stability:

- **Batch Normalization:** Running mean/variance updated in FP32.
- **Softmax:** Computed in FP32 to avoid overflow.
- **Loss Computation:** Always FP32.
- **Master Weights:** Stored in FP32; cast to FP16 for forward/backward.

4.2 Exponential Moving Average (EMA)

EMA smooths parameter updates, leading to better generalization (?).

4.2.1 EMA Update Rule

Maintain shadow parameters θ_{EMA} :

$$\theta_{\text{EMA}}^{(t+1)} = \beta \theta_{\text{EMA}}^{(t)} + (1 - \beta) \theta^{(t+1)}, \quad (4.3)$$

where $\beta \in (0, 1)$ is the decay rate (typically $\beta = 0.995$ or $\beta = 0.9999$).

4.2.2 Bias Correction

During early training, EMA is biased toward initialization. Apply bias correction:

$$\hat{\theta}_{\text{EMA}}^{(t)} = \frac{\theta_{\text{EMA}}^{(t)}}{1 - \beta^t}. \quad (4.4)$$

4.2.3 EMA for Batch Normalization

Batch normalization layers maintain running statistics. Apply EMA separately:

$$\mu_{\text{running}}^{(t+1)} = \beta_{\text{BN}} \mu_{\text{running}}^{(t)} + (1 - \beta_{\text{BN}}) \mu_{\text{batch}}^{(t)}, \quad (4.5)$$

$$\sigma_{\text{running}}^{2(t+1)} = \beta_{\text{BN}} \sigma_{\text{running}}^{2(t)} + (1 - \beta_{\text{BN}}) \sigma_{\text{batch}}^{2(t)}, \quad (4.6)$$

where $\beta_{\text{BN}} \approx 0.9$ (faster decay than model weights).

4.2.4 EMA Benefits for Diffusion Models

1. **Improved Sample Quality:** EMA parameters often produce higher-quality samples than raw parameters.
2. **Training Stability:** Smoothing reduces variance from stochastic gradient noise.
3. **Reduced Overfitting:** EMA acts as implicit regularization.

4.3 Gradient Accumulation

When memory constraints limit batch size, gradient accumulation simulates larger batches:

Algorithm 4 Gradient Accumulation

- 1: **Input:** Model θ , micro-batch size B_{micro} , accumulation steps K
 - 2: **Output:** Updated model θ
 - 3: Initialize $\mathbf{g}_{\text{accum}} = \mathbf{0}$
 - 4: **for** $k = 1$ to K **do**
 - 5: Sample micro-batch $\mathcal{B}_k$ of size B_{micro}
 - 6: Compute loss $\mathcal{L}_k$ on $\mathcal{B}_k$
 - 7: Compute gradients $\mathbf{g}_k = \nabla_{\theta} \mathcal{L}_k$
 - 8: $\mathbf{g}_{\text{accum}} \leftarrow \mathbf{g}_{\text{accum}} + \mathbf{g}_k$
 - 9: **end for**
 - 10: $\mathbf{g}_{\text{accum}} \leftarrow \mathbf{g}_{\text{accum}} / K$
 - 11: Update $\theta \leftarrow \theta - \eta \mathbf{g}_{\text{accum}}$
-

Effective batch size: $B_{\text{eff}} = K \cdot B_{\text{micro}}$.

4.4 Optimizer Selection

4.4.1 AdamW

AdamW (?) decouples weight decay from gradient-based updates:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (4.7)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (4.8)$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t), \quad (4.9)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda \theta_{t-1} \right), \quad (4.10)$$

where λ is the weight decay coefficient.

4.4.2 Learning Rate Schedules

Cosine Annealing

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T} \pi \right) \right), \quad (4.11)$$

where t is the current step and T is the total number of steps.

Warmup

Linear warmup for the first T_{warmup} steps:

$$\eta_t = \begin{cases} \frac{t}{T_{\text{warmup}}} \eta_{\max}, & t \leq T_{\text{warmup}}, \\ \eta_t^{(\text{schedule})}, & t > T_{\text{warmup}}. \end{cases} \quad (4.12)$$

Chapter 5

Model Architecture

This chapter describes the conditional U-Net architecture employed in our compact diffusion model, with emphasis on design choices for parameter efficiency and computational performance.

5.1 U-Net Architecture Overview

The U-Net (?) is an encoder-decoder architecture with skip connections, originally designed for image segmentation but widely adopted for diffusion models (?).

5.1.1 Encoder-Decoder Structure

Encoder

The encoder progressively downsamples spatial dimensions while increasing channel capacity:

$$\mathbf{h}^{(\ell)} = \text{Down}^{(\ell)}(\mathbf{h}^{(\ell-1)}, \mathbf{e}_t), \quad (5.1)$$

where $\mathbf{h}^{(\ell)} \in \mathbb{R}^{C_\ell \times H_\ell \times W_\ell}$ with $H_\ell = H_{\ell-1}/2$ and $W_\ell = W_{\ell-1}/2$.

Decoder

The decoder upsamples features, incorporating skip connections from corresponding encoder layers:

$$\mathbf{h}^{(\ell)} = \text{Up}^{(\ell)}(\mathbf{h}^{(\ell+1)}, \mathbf{h}_{\text{skip}}^{(L-\ell)}, \mathbf{e}_t), \quad (5.2)$$

where $\mathbf{h}_{\text{skip}}^{(L-\ell)}$ is the corresponding encoder feature.

5.2 Building Blocks

5.2.1 Double Convolution Block

The fundamental building block is a sequence of two convolutions with normalization and activation:

Figure 5.1: Complete U-Net architecture showing encoder, bottleneck, and decoder paths with skip connections.

$$\mathbf{z}_1 = \text{GELU}(\text{GroupNorm}(\text{Conv}(\mathbf{x}))), \quad (5.3)$$

$$\mathbf{z}_2 = \text{GELU}(\text{GroupNorm}(\text{Conv}(\mathbf{z}_1))). \quad (5.4)$$

With residual connection:

$$\mathbf{y} = \mathbf{x} + \mathbf{z}_2, \quad (5.5)$$

where channel dimensions are matched via 1×1 convolution if necessary.

5.2.2 Self-Attention Layer

Self-attention enables modeling long-range dependencies:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (5.6)$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are linear projections of the input features.

Figure 5.2: ResNet-style convolutional block with skip connection.

Figure 5.3: Self-attention block with layer normalization and feed-forward network.

Multi-Head Attention

Split into h heads:

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad (5.7)$$

where $\text{head}_i = \text{Attention}(\mathbf{XW}_i^Q, \mathbf{XW}_i^K, \mathbf{XW}_i^V)$.

5.2.3 Downsampling and Upsampling

Downsampling

Combine max pooling with strided convolution:

$$\mathbf{h}_{\text{pool}} = \text{MaxPool}(\mathbf{h}^{(\ell-1)}), \quad (5.8)$$

$$\mathbf{h}^{(\ell)} = \text{DoubleConv}(\mathbf{h}_{\text{pool}}, \mathbf{e}_t). \quad (5.9)$$

Upsampling

Use bilinear interpolation followed by convolution:

block.png

Figure 5.4: Upsampling block with bilinear interpolation and convolution.

$$\mathbf{h}_{\text{up}} = \text{Upsample}(\mathbf{h}^{(\ell+1)}, \text{scale} = 2), \quad (5.10)$$

$$\mathbf{h}_{\text{concat}} = \text{Concat}(\mathbf{h}_{\text{up}}, \mathbf{h}_{\text{skip}}^{(L-\ell)}), \quad (5.11)$$

$$\mathbf{h}^{(\ell)} = \text{DoubleConv}(\mathbf{h}_{\text{concat}}, \mathbf{e}_t). \quad (5.12)$$

5.3 Conditioning Mechanisms

5.3.1 Timestep Embedding

The diffusion timestep t is embedded using sinusoidal functions:

$$\text{PE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{PE}(t, 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right), \quad (5.13)$$

where d is the embedding dimension.

These embeddings are passed through MLPs to match feature dimensions:

$$\mathbf{e}_t = \text{GELU}(\mathbf{W}_2 \text{GELU}(\mathbf{W}_1 \text{PE}(t))). \quad (5.14)$$

5.3.2 Class Conditioning

For class-conditional generation, class labels $y \in \{1, \dots, C\}$ are embedded:

$$\mathbf{e}_y = \text{Embedding}(y), \quad (5.15)$$

and added to the timestep embedding:

$$\mathbf{e} = \mathbf{e}_t + \mathbf{e}_y. \quad (5.16)$$

5.3.3 Adaptive Group Normalization (AdaGN)

Conditioning information modulates normalization statistics:

$$\text{AdaGN}(\mathbf{h}, \mathbf{e}) = \gamma(\mathbf{e}) \cdot \frac{\mathbf{h} - \mu}{\sigma} + \beta(\mathbf{e}), \quad (5.17)$$

where γ and β are predicted from $\mathbf{e}$ via linear layers.

5.4 Compact Design Choices

5.4.1 Channel Compression Factor

Introduce compression factor $c \in (0, 1]$:

$$C_\ell^{\text{compact}} = \lfloor c \cdot C_\ell^{\text{base}} \rfloor, \quad (5.18)$$

reducing parameter count by approximately c^2 (due to convolutional weight matrices).

5.4.2 Attention Placement

Attention is computationally expensive ($O(N^2)$ where $N = H \times W$). Apply attention only at low resolutions:

- Resolution $\geq 32 \times 32$: No attention
- Resolution 16×16 : Single attention layer
- Resolution 8×8 : Multiple attention layers

5.4.3 Reduced Attention Heads

Use fewer attention heads ($h = 4$ instead of $h = 8$) in compact models.

5.4.4 Depthwise Separable Convolutions

Replace standard convolutions with depthwise separable convolutions:

$$\text{DepthwiseConv}(\mathbf{x}) = \text{Conv}_{3 \times 3}^{\text{depthwise}}(\mathbf{x}), \quad (5.19)$$

$$\text{PointwiseConv}(\mathbf{y}) = \text{Conv}_{1 \times 1}(\mathbf{y}), \quad (5.20)$$

reducing parameters from $C_{\text{in}} \cdot C_{\text{out}} \cdot k^2$ to $C_{\text{in}} \cdot k^2 + C_{\text{in}} \cdot C_{\text{out}}$.

Chapter 6

Symmetry-Aware Diffusion Models

This chapter develops a comprehensive framework for incorporating physical symmetries and geometric invariances into diffusion model architectures. We draw from principles in group theory, geometric deep learning (?), and physics-informed machine learning.

6.1 Motivation and Mathematical Foundations

6.1.1 Symmetries in Physical Systems

Physical systems often exhibit symmetries corresponding to transformations that leave the system's properties invariant. Examples include:

Translation Invariance Physics laws are the same everywhere in space.

Rotational Invariance Isotropic systems have the same properties in all directions.

Permutation Symmetry Identical particles are indistinguishable.

Gauge Symmetry Physical observables are invariant under local phase transformations.

Incorporating these symmetries as inductive biases offers several advantages:

1. **Data Efficiency:** Fewer training examples required to learn symmetry-respecting functions.
2. **Generalization:** Better performance on out-of-distribution data respecting the same symmetries.
3. **Interpretability:** Generated samples automatically satisfy physical constraints.
4. **Parameter Efficiency:** Weight sharing induced by equivariance reduces model capacity needs.

6.1.2 Group Theory Fundamentals

Definition 6.1.1 (Group). A group $(G, \cdot)$ is a set G equipped with a binary operation $\cdot : G \times G \rightarrow G$ satisfying:

1. Closure: $\forall g_1, g_2 \in G, g_1 \cdot g_2 \in G$.

2. Associativity: $(g_1 \cdot g_2) \cdot g_3 = g_1 \cdot (g_2 \cdot g_3)$.
3. Identity: $\exists e \in G$ such that $e \cdot g = g \cdot e = g$ for all $g \in G$.
4. Inverse: $\forall g \in G, \exists g^{-1} \in G$ such that $g \cdot g^{-1} = g^{-1} \cdot g = e$.

Definition 6.1.2 (Group Action). A (left) action of group G on set X is a function $\rho : G \times X \rightarrow X$ satisfying:

1. $\rho(e, x) = x$ for all $x \in X$,
2. $\rho(g_1, \rho(g_2, x)) = \rho(g_1 \cdot g_2, x)$ for all $g_1, g_2 \in G, x \in X$.

We often write $\rho(g, x) = g \cdot x$ or $T_g(x)$ where $T_g : X \rightarrow X$ is the transformation induced by g .

6.1.3 Equivariance and Invariance

Definition 6.1.3 (Equivariance). A function $f : X \rightarrow Y$ is equivariant with respect to group actions ρ_X on X and ρ_Y on Y if:

$$f(\rho_X(g, x)) = \rho_Y(g, f(x)), \quad \forall g \in G, x \in X. \quad (6.1)$$

Equivalently: $f \circ T_g^X = T_g^Y \circ f$.

Definition 6.1.4 (Invariance). A function $f : X \rightarrow Y$ is invariant if ρ_Y is the trivial action (identity):

$$f(\rho_X(g, x)) = f(x), \quad \forall g \in G, x \in X. \quad (6.2)$$

6.1.4 Relevant Symmetry Groups

Translation Group $\mathbb{R}^d$

Group operation: vector addition. Action on functions:

$$(T_{\mathbf{v}}f)(\mathbf{x}) = f(\mathbf{x} - \mathbf{v}). \quad (6.3)$$

Convolutional networks are naturally translation equivariant.

Rotation Group $SO(d)$

Special orthogonal group in d dimensions. Action on points:

$$T_R(\mathbf{x}) = R\mathbf{x}, \quad R \in SO(d). \quad (6.4)$$

Euclidean Group $E(d) = \mathbb{R}^d \rtimes SO(d)$

Semidirect product of translations and rotations:

$$T_{(\mathbf{v}, R)}(\mathbf{x}) = R\mathbf{x} + \mathbf{v}. \quad (6.5)$$

Permutation Group S_N

Group of permutations on N elements. Action on sequences:

$$T_\pi(\mathbf{x}_1, \dots, \mathbf{x}_N) = (\mathbf{x}_{\pi(1)}, \dots, \mathbf{x}_{\pi(N)}). \quad (6.6)$$

Relevant for point clouds, molecular systems, particle physics.

6.2 Equivariant Neural Networks

6.2.1 Group Convolution

Definition 6.2.1 (Group Convolution). For group G acting on feature space X , the group convolution is:

$$[f \star \psi](g) = \int_G f(h) \psi(h^{-1}g) dh, \quad (6.7)$$

where $f : G \rightarrow \mathbb{R}^{C_{\text{in}}}$ is the input feature map and $\psi : G \rightarrow \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$ is the filter.

For discrete groups, the integral becomes a sum.

6.2.2 Steerable Convolutions

For continuous groups like $SO(2)$ or $SO(3)$, filters can be constructed from steerable bases (?):

$$\psi(\mathbf{r}, R) = \sum_{\ell=0}^{L_{\max}} \sum_{m=-\ell}^{\ell} c_{\ell m} Y_{\ell}^m(R\mathbf{r}), \quad (6.8)$$

where Y_{ℓ}^m are spherical harmonics and $c_{\ell m}$ are learnable coefficients.

6.2.3 $E(3)$ -Equivariant Graph Networks

For molecular and particle data represented as graphs, Satorras et al. (?) proposed $E(n)$ -equivariant graph networks:

$$\mathbf{m}_{ij} = \phi_m(\mathbf{h}_i, \mathbf{h}_j, \|\mathbf{x}_i - \mathbf{x}_j\|^2), \quad (6.9)$$

$$\mathbf{x}'_i = \mathbf{x}_i + \sum_{j \neq i} (\mathbf{x}_i - \mathbf{x}_j) \phi_x(\mathbf{m}_{ij}), \quad (6.10)$$

$$\mathbf{h}'_i = \phi_h(\mathbf{h}_i, \sum_{j \neq i} \mathbf{m}_{ij}), \quad (6.11)$$

where ϕ_m, ϕ_x, ϕ_h are MLPs, $\mathbf{x}_i \in \mathbb{R}^3$ are coordinates, and $\mathbf{h}_i \in \mathbb{R}^C$ are features. Crucially, the coordinate update (Equation 6.10) is equivariant:

$$T_R(\mathbf{x}'_i) = R\mathbf{x}_i + \sum_{j \neq i} R(\mathbf{x}_i - \mathbf{x}_j) \phi_x(\mathbf{m}_{ij}) = R\mathbf{x}'_i. \quad (6.12)$$

6.3 Symmetry-Aware Diffusion Models

6.3.1 Equivariant Forward Process

For data $\mathbf{x}_0$ transforming under group G , the forward diffusion should respect symmetry:

$$q(T_g(\mathbf{x}_t)|T_g(\mathbf{x}_{t-1})) = q(\mathbf{x}_t|\mathbf{x}_{t-1}). \quad (6.13)$$

For Gaussian diffusion:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}), \quad (6.14)$$

this is automatically satisfied for $G = \mathbb{R}^d$ (translations) and $G = SO(d)$ (rotations), since the Gaussian is isotropic.

6.3.2 Equivariant Denoising Network

The key requirement is that the denoising function $\epsilon_\theta(\mathbf{x}_t, t)$ is equivariant:

$$\epsilon_\theta(T_g(\mathbf{x}_t), t) = T_g(\epsilon_\theta(\mathbf{x}_t, t)). \quad (6.15)$$

Implementation via Equivariant Architectures

Replace standard U-Net layers with group-equivariant analogs:

- **Translation Equivariance:** Standard convolutions.
- **Rotation Equivariance:** Use steerable CNNs (?) or vector field networks (?).
- **Permutation Equivariance:** Use graph networks or DeepSets (?).
- **$E(3)$ Equivariance:** Use EGNN (?) or Tensor Field Networks (?).

6.3.3 Score Function Equivariance

From the score-based perspective, equivariance of $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ is natural:

Proposition 6.3.1 (Score Equivariance). *If $p(T_g(\mathbf{x})) = p(\mathbf{x})$ for all $g \in G$ (density is invariant), then:*

$$\nabla_{\mathbf{x}} \log p(T_g(\mathbf{x})) = J_g^\top \nabla_{T_g(\mathbf{x})} \log p(T_g(\mathbf{x})), \quad (6.16)$$

where $J_g = \frac{\partial T_g}{\partial \mathbf{x}}$ is the Jacobian.

For linear actions ($T_g(\mathbf{x}) = g\mathbf{x}$ with g a matrix), this simplifies to:

$$\nabla_{\mathbf{x}} \log p(g\mathbf{x}) = g^\top \nabla_{g\mathbf{x}} \log p(g\mathbf{x}). \quad (6.17)$$

6.3.4 Gauge Equivariance

For systems with gauge symmetry (e.g., molecular conformations invariant to global rotations), we can parameterize in a gauge-invariant coordinate system.

Internal Coordinates

For molecular systems, use bond lengths, angles, and dihedrals:

$$d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|, \quad (6.18)$$

$$\theta_{ijk} = \arccos \left(\frac{(\mathbf{x}_i - \mathbf{x}_j) \cdot (\mathbf{x}_k - \mathbf{x}_j)}{d_{ij}d_{jk}} \right), \quad (6.19)$$

$$\phi_{ijkl} = \text{dihedral}(\mathbf{x}_i, \mathbf{x}_j, \mathbf{x}_k, \mathbf{x}_l). \quad (6.20)$$

Diffusion operates in this internal coordinate space, ensuring $SE(3)$ invariance automatically.

6.4 Incorporating Conservation Laws

6.4.1 Continuous Symmetries and Noether's Theorem

Noether's theorem states that every continuous symmetry corresponds to a conserved quantity:

Theorem 6.4.1 (Noether's Theorem). *If a Lagrangian $\mathcal{L}$ is invariant under a continuous transformation parameterized by ϵ :*

$$\mathbf{x}(t) \rightarrow \mathbf{x}(t) + \epsilon \delta \mathbf{x}(t), \quad (6.21)$$

then there exists a conserved charge:

$$Q = \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{x}}} \cdot \delta \mathbf{x}. \quad (6.22)$$

Examples:

- Translation symmetry $\Rightarrow$ momentum conservation.
- Rotation symmetry $\Rightarrow$ angular momentum conservation.
- Time translation symmetry $\Rightarrow$ energy conservation.

6.4.2 Constrained Diffusion

To enforce conservation laws, we can modify the diffusion process to operate on a constrained manifold (?).

Projection Method

After each diffusion step, project onto the constraint manifold:

$$\mathbf{x}'_t = \arg \min_{\mathbf{x} \in \mathcal{M}} \|\mathbf{x} - \mathbf{x}_t\|^2, \quad (6.23)$$

where $\mathcal{M} = \{\mathbf{x} : Q(\mathbf{x}) = Q_0\}$ is the constraint manifold.

Geodesic Diffusion

For constraints defining a Riemannian manifold, diffusion can occur along geodesics:

$$d\mathbf{x} = -\nabla_{\mathcal{M}} \log p_t(\mathbf{x}) dt + \sqrt{2} d\mathbf{w}_{\mathcal{M}}, \quad (6.24)$$

where $\nabla_{\mathcal{M}}$ is the Riemannian gradient and $d\mathbf{w}_{\mathcal{M}}$ is Brownian motion on $\mathcal{M}$.

6.4.3 Energy-Based Models and Hamiltonian Dynamics

For systems governed by Hamiltonian dynamics $H(\mathbf{q}, \mathbf{p})$, we can define diffusion in phase space:

$$d\mathbf{q} = \frac{\partial H}{\partial \mathbf{p}} dt + \sigma_q d\mathbf{w}_q, \quad (6.25)$$

$$d\mathbf{p} = -\frac{\partial H}{\partial \mathbf{q}} dt + \sigma_p d\mathbf{w}_p. \quad (6.26)$$

The denoising network learns:

$$\epsilon_{\theta}(\mathbf{q}, \mathbf{p}, t) = \left(\frac{\partial H_{\theta}}{\partial \mathbf{p}}, -\frac{\partial H_{\theta}}{\partial \mathbf{q}} \right), \quad (6.27)$$

ensuring generated trajectories respect energy conservation (up to noise).

6.5 Symmetry-Induced Model Compression

6.5.1 Parameter Sharing via Equivariance

Equivariant networks naturally share parameters across group orbits, reducing effective capacity.

Proposition 6.5.1 (Parameter Count Reduction). *For a group G acting on input/output spaces, a G -equivariant linear layer has parameter count:*

$$|\Theta_{equiv}| = \frac{|\Theta_{standard}|}{|G|}, \quad (6.28)$$

where $|G|$ is the group order (for finite groups).

For continuous groups, the reduction factor depends on the representation dimensionality.

6.5.2 Factorized Generation

For systems with independent symmetry sectors, generation can be factorized:

$$p(\mathbf{x}) = \prod_{i=1}^K p(\mathbf{x}^{(i)}), \quad (6.29)$$

where $\mathbf{x}^{(i)}$ are disjoint components transformed by subgroups $G_i \subseteq G$.

Train separate compact models for each sector:

$$\epsilon_{\theta}(\mathbf{x}_t, t) = \bigoplus_{i=1}^K \epsilon_{\theta_i}(\mathbf{x}_t^{(i)}, t). \quad (6.30)$$

Total parameter count: $\sum_i |\theta_i| < |\theta_{\text{monolithic}}|$.

6.6 Examples and Applications

6.6.1 Image Generation with Rotation Equivariance

For natural images exhibiting approximate rotational symmetry (e.g., flowers, wheels):

Use $SO(2)$ -equivariant convolutions (?). Filter bank:

$$\psi^{(k)}(r, \theta) = \sum_{\ell=0}^{L_{\max}} c_{\ell}^{(k)} R_{\ell}(r) e^{i\ell\theta}, \quad (6.31)$$

where $R_{\ell}(r)$ are radial functions and $c_{\ell}^{(k)}$ are learned coefficients.

6.6.2 Molecular Conformation Generation

For molecule generation respecting $SE(3)$ symmetry (?):

1. Represent molecule as graph: nodes = atoms, edges = bonds.
2. Use EGNN layers (Equations 6.9–6.11) for ϵ_{θ} .
3. Diffusion operates on 3D coordinates $\{\mathbf{x}_i\}$ and node features $\{\mathbf{h}_i\}$.
4. Conservation: enforce center of mass at origin, optionally constrain bond lengths.

6.6.3 Point Cloud Generation

For 3D point clouds (e.g., LiDAR, CAD):

Use permutation-equivariant architectures like PointNet++ (?) or DGCNN (?) as the backbone for ϵ_{θ} .

Diffusion on point clouds:

$$\mathbf{P}_t = \{\mathbf{x}_i^{(t)}\}_{i=1}^N, \quad \mathbf{x}_i^{(t)} = \sqrt{\bar{\alpha}_t} \mathbf{x}_i^{(0)} + \sqrt{1 - \bar{\alpha}_t} \epsilon_i. \quad (6.32)$$

Denoising network must be permutation equivariant:

$$\epsilon_{\theta}(T_{\pi}(\mathbf{P}_t), t) = T_{\pi}(\epsilon_{\theta}(\mathbf{P}_t, t)). \quad (6.33)$$

6.6.4 Physics Simulation

For particle systems governed by pairwise potentials $V(\mathbf{x}) = \sum_{i < j} v(\|\mathbf{x}_i - \mathbf{x}_j\|)$:

Score function:

$$\nabla_{\mathbf{x}_i} \log p(\mathbf{x}) = -\nabla_{\mathbf{x}_i} V(\mathbf{x}) = -\sum_{j \neq i} v'(\|\mathbf{x}_i - \mathbf{x}_j\|) \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|}. \quad (6.34)$$

This is naturally $E(3)$ -equivariant. Neural network learns the potential:

$$v_{\theta}(r) \approx v(r). \quad (6.35)$$

6.7 Combining Symmetry with Other Compression Techniques

6.7.1 Symmetry-Aware Pruning

Pruning must respect equivariance constraints. For group-equivariant layers, prune entire irreducible representations (irreps) rather than individual channels:

$$\text{Prune irrep } \rho_\ell \text{ if } \text{importance}(\rho_\ell) < \delta, \quad (6.36)$$

where importance aggregates across all channels in the irrep.

6.7.2 Symmetry-Aware Quantization

Quantization scales can be shared across group orbits:

$$\Delta_g = \Delta \quad \text{for all } g \in G, \quad (6.37)$$

reducing the number of quantization parameters.

6.7.3 Distillation with Symmetry Constraints

Student and teacher must both respect the same symmetries. Distillation loss can include symmetry-specific terms:

$$\mathcal{L}_{\text{sym}} = \mathbb{E}_{g \sim G} [\|\epsilon_S(T_g(\mathbf{x}_t), t) - T_g(\epsilon_S(\mathbf{x}_t, t))\|^2], \quad (6.38)$$

explicitly penalizing symmetry violations.

Chapter 7

Experimental Results and Analysis

This chapter presents our experimental methodology, quantitative results, and critical analysis of the compact diffusion models developed in this work.

7.1 Experimental Setup

7.1.1 Dataset

We use the CIFAR-10 dataset (?), consisting of 60,000 color images (32×32 pixels) across 10 classes. Due to computational constraints:

- **Training Set:** 10,000 randomly sampled images.
- **Validation Set:** 1,000 images.
- **Test Set:** 1,000 images.

Images are resized to 64×64 pixels and normalized to $[-1, 1]$.

7.1.2 Model Configuration

Teacher Model

- **Architecture:** U-Net with base channels $C = 128$, compression factor $c = 1$.
- **Timesteps:** $T = 1000$, linear variance schedule $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$.
- **Self-Attention:** At resolutions 16×16 and 8×8 , 4 heads.

Student Model

- **Architecture:** U-Net with base channels $C = 64$, compression factor $c = 0.5$.
- **Distillation:** Combined loss with $\alpha = 0.5$, $\beta = 0.5$ (Equation 3.4).
- **Feature Matching:** 3 intermediate layers, $\gamma_\ell = 0.1$.

Pruned Student Model

- **Pruning Method:** Structured channel pruning with L1 norm importance.
- **Pruning Ratio:** Global 30% sparsity.
- **Fine-Tuning:** 50 epochs post-pruning.

7.1.3 Training Details

- **Optimizer:** AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay $\lambda = 10^{-4}$.
- **Learning Rate:** Fixed $\eta = 10^{-5}$ (suboptimal; see discussion).
- **Batch Size:** 64 (with gradient accumulation to simulate batch size 128).
- **Epochs:** Approximately 200 (limited by compute).
- **Mixed Precision:** FP16 with dynamic loss scaling.
- **EMA:** Decay $\beta = 0.995$, applied after 1000 warmup steps.

7.1.4 Hardware

Single NVIDIA V100 GPU (16GB VRAM), training time approximately 48 hours for teacher, 36 hours for student.

7.2 Quantitative Results

7.2.1 Model Size and Inference Time

Model	Parameters (M)	Size (MB)	Inference Time (s/batch)
Teacher	23.5	89.02	18.54
Student (Distilled)	6.0	22.60	15.72
Student (Pruned 30%)	4.6	17.42	15.40

Table 7.1: Model size and inference time comparison. Batch size = 8, $T = 1000$ sampling steps.

Key Observations:

- Student achieves 74.6% size reduction compared to teacher.
- Pruning provides additional 23% reduction (15% relative to student).
- Inference speedup modest (15%) due to memory-bound operations; more significant speedup expected on mobile/edge devices.

7.2.2 Generation Quality: FID Scores

Model	FID ↓	Inception Score ↑	Training Epochs
Teacher	163.2	3.41	200
Student (Distilled)	184.7	3.12	200
Student (Pruned 30%)	213.5	2.85	200 + 50 fine-tune

Table 7.2: Generation quality metrics. Lower FID is better; higher Inception Score is better. For reference, fully-trained DDPM on full CIFAR-10 achieves FID ≈ 3 –5.

Key Observations:

- All models exhibit elevated FID scores, indicating significant deviation from real data distribution.
- Student suffers 13% FID degradation relative to teacher.
- Aggressive pruning causes additional 16% FID degradation.

7.2.3 Training Curves

Figure 7.1: MSE training loss for teacher model over 200 epochs. Loss decreases steadily but has not fully converged.

Figure 7.2: MSE training loss for student model with knowledge distillation. Combined loss shown; standard diffusion loss and distillation loss behave similarly.

7.2.4 Generated Samples

Figure 7.3: Sample images generated by the teacher model after 200 epochs. Recognizable class features but lacking fine details and exhibiting noise artifacts.

Figure 7.4: Sample images generated by the distilled student model. Quality comparable to teacher but with slightly reduced sharpness.

Figure 7.5: Sample images generated by the pruned student model (30% sparsity). Visible quality degradation with increased noise and loss of fine details.

7.3 Critical Analysis

7.3.1 Factors Contributing to High FID Scores

Insufficient Training

Our models were trained for only 200 epochs on 10,000 images. State-of-the-art diffusion models typically require:

- **Epochs:** 500–1000+ epochs
- **Data:** Full datasets (50,000–millions of images)
- **Compute:** Multiple high-end GPUs for days/weeks

The training curves (Figures 7.1, 7.2) show continued decrease, indicating lack of convergence.

Fixed Learning Rate

Using a fixed learning rate $\eta = 10^{-5}$ throughout training is suboptimal. Adaptive schedules (cosine annealing, warm restarts) enable:

- Initial rapid learning with higher rates
- Fine-grained optimization with decreased rates near convergence
- Escape from local minima via periodic rate increases

Estimated Impact: Could reduce FID by 30–50 points.

Limited Dataset

Training on 10,000 images (16.7% of CIFAR-10) severely limits:

- Diversity of learned features
- Generalization to unseen modes
- Coverage of intra-class variations

Data augmentation (random crops, flips, color jitter) was minimal.

Estimated Impact: Could reduce FID by 40–60 points.

Aggressive Compression

The student model has 74% fewer parameters than the teacher. While knowledge distillation mitigates capacity reduction, some performance loss is inevitable. The pruned model (30% sparsity) removes critical connections, as evidenced by the FID degradation from 184.7 to 213.5.

7.3.2 Comparison to Baselines

Model	Dataset	Params (M)	FID
DDPM (?)	CIFAR-10 (full)	35.7	3.17
Improved DDPM (?)	CIFAR-10 (full)	35.7	2.90
Our Teacher	CIFAR-10 (10k subset)	23.5	163.2
Our Student	CIFAR-10 (10k subset)	6.0	184.7

Table 7.3: Comparison to published baselines. Note: direct comparison is unfair due to different training regimes.

The large FID gap (160+ points) is primarily attributable to training regime differences rather than architectural deficiencies.

7.3.3 Distillation Effectiveness

Despite the absolute FID scores being high, the *relative* performance is informative:

- Student achieves 74% size reduction with only 13% FID degradation (184.7 vs 163.2).
- This corresponds to a compression-quality trade-off of $\Delta\text{FID}/\Delta\text{size} \approx 0.24$ FID points per MB saved.

Interpretation: Knowledge distillation successfully transfers learned representations despite capacity constraints.

7.3.4 Pruning Trade-offs

Pruning 30% of channels yields:

- **Benefit:** 23% size reduction, minor inference speedup
- **Cost:** 16% FID degradation (213.5 vs 184.7)

Analysis: The FID increase indicates that the pruned channels contained non-redundant information. Less aggressive pruning (e.g., 15–20%) or iterative pruning with longer fine-tuning would likely improve the trade-off.

7.4 Ablation Studies

7.4.1 Impact of Distillation Loss Components

Configuration	FID	α	β
Standard diffusion loss only	198.3	1.0	0.0
Output distillation only	192.1	0.0	1.0
Combined ($\alpha = \beta = 0.5$)	184.7	0.5	0.5
Combined + feature matching	182.4	0.5	0.5

Table 7.4: Ablation study on distillation loss components. Feature matching provides small but consistent improvement.

7.4.2 Impact of EMA

Configuration	FID (raw weights)	FID (EMA weights)
Teacher	171.5	163.2
Student	192.8	184.7

Table 7.5: Effect of using EMA weights for evaluation. EMA consistently improves FID by 8–10 points.

7.5 Limitations

7.5.1 Computational Constraints

- **Training Time:** Limited to 48 hours per model due to hardware availability.
- **Hyperparameter Search:** No systematic search; parameters chosen based on prior work and manual tuning.
- **Ensemble Models:** Could not train multiple runs for statistical significance.

7.5.2 Architectural Choices

- **Attention Layers:** Limited to low resolutions; full-resolution attention might improve quality.
- **Depth:** Relatively shallow U-Net; deeper architectures could learn richer representations.
- **Advanced Modules:** Did not explore recent innovations like cross-attention, adaptive layer norm, or flash attention.

7.5.3 Evaluation Metrics

- **FID:** Sensitive to sample size and feature extractor (Inception-v3); not always aligned with perceptual quality.
- **Human Evaluation:** No user studies conducted; perceptual metrics (LPIPS) could provide complementary insights.
- **Diversity:** Did not explicitly measure mode coverage or diversity (e.g., via precision/recall metrics).

Chapter 8

Conclusion and Future Directions

8.1 Summary of Contributions

This work has presented a comprehensive theoretical and empirical study of compact diffusion models, with the following key contributions:

1. **Rigorous Mathematical Foundations:** Complete derivations of diffusion model training objectives, including variational lower bounds, score-based formulations, and SDE perspectives.
2. **Compression Techniques:** Detailed analysis of knowledge distillation, structured pruning, and quantization for diffusion models, with novel formulations for timestep-aware importance metrics and progressive distillation.
3. **Symmetry-Aware Framework:** Introduction of a comprehensive framework for incorporating physical symmetries into diffusion architectures, connecting geometric deep learning to generative modeling.
4. **Empirical Validation:** Demonstration of 74% model size reduction with controlled performance degradation through knowledge distillation and pruning.

8.2 Key Insights

- **Distillation Efficacy:** Knowledge distillation successfully transfers learned representations from teacher to student, maintaining functional performance (as measured by training loss) despite significant capacity reduction.
- **Training Regime Criticality:** The quality of diffusion models is highly sensitive to training duration, learning rate schedules, and dataset size. Our elevated FID scores primarily reflect insufficient training rather than architectural deficiencies.
- **Pruning Trade-offs:** Structured channel pruning provides marginal inference speedup but can significantly degrade generation quality if applied too aggressively. Iterative pruning with extensive fine-tuning is essential.
- **Symmetry Benefits:** Incorporating symmetries as inductive biases offers a promising avenue for improving both parameter efficiency and data efficiency, though implementation requires careful architectural design.

8.3 Future Research Directions

8.3.1 Improved Training Protocols

1. **Extended Training:** Train on full CIFAR-10 (or larger datasets like ImageNet) for 500–1000 epochs.
2. **Adaptive Learning Rates:** Implement cosine annealing with warm restarts; explore learning rate warmup.
3. **Data Augmentation:** Extensive augmentation strategies (AutoAugment, RandAugment) to improve generalization with limited data.
4. **Large Batch Training:** Utilize gradient accumulation and distributed training for effective batch sizes of 256–512.

8.3.2 Advanced Compression

1. **Quantization Implementation:** Complete quantization-aware training pipeline with per-channel quantization and dynamic bit-width allocation.
2. **Progressive Distillation:** Implement full progressive distillation (?) to reduce sampling steps from 1000 to 4–8.
3. **Consistency Models:** Explore consistency distillation (?) for single-step generation.
4. **Hybrid Methods:** Combine multiple compression techniques (e.g., quantization + pruning + distillation) with careful coordination.

8.3.3 Latent Diffusion Models

Transition to latent diffusion (?) for significantly improved efficiency:

- Train variational autoencoder (VAE) to compress images to low-dimensional latent space.
- Perform diffusion in latent space (e.g., $4 \times 4 \times C$ instead of $64 \times 64 \times 3$).
- Expected benefits: 8–16× speedup, improved sample quality, reduced memory footprint.

8.3.4 Symmetry-Aware Architectures

1. **$SO(2)$ Equivariant U-Nets:** Implement steerable CNNs for rotation-equivariant image generation.
2. **Point Cloud Diffusion:** Develop permutation-equivariant diffusion models for 3D point clouds using PointNet++ or DGCNN backbones.
3. **Molecular Generation:** Apply $E(3)$ -equivariant graph networks for drug discovery and materials science applications.
4. **Physics Simulation:** Incorporate conservation laws (energy, momentum, angular momentum) via constrained diffusion on Riemannian manifolds.

8.3.5 Theoretical Analysis

1. **Convergence Guarantees:** Formal analysis of convergence rates for distilled and pruned models.
2. **Capacity Bounds:** Determine minimum model capacity required to approximate teacher performance within ϵ error.
3. **Symmetry Breaking:** Study conditions under which learned representations spontaneously break or preserve symmetries.
4. **Score Approximation:** Error bounds for score function approximation in equivariant networks.

8.3.6 Application Domains

1. **Medical Imaging:** Compact diffusion models for MRI/CT image synthesis with privacy-preserving constraints.
2. **Scientific Computing:** Accelerated sampling for molecular dynamics, weather forecasting, and computational fluid dynamics.
3. **Edge Deployment:** Mobile/IoT implementation with hardware-aware neural architecture search.
4. **Controllable Generation:** Integrate symmetry-aware diffusion with conditional generation (text-to-image, image editing).

8.4 Broader Impact

Developing compact generative models has significant implications:

8.4.1 Positive Impacts

- **Democratization:** Enabling generative AI on consumer hardware, reducing barriers to entry.
- **Energy Efficiency:** Reduced computational requirements lead to lower carbon footprint.
- **Privacy:** On-device generation avoids data transmission to centralized servers.
- **Scientific Discovery:** Accelerating simulation-based inference in physics, chemistry, and biology.

8.4.2 Potential Concerns

- **Misuse:** Compact models could facilitate creation of deepfakes or disinformation at scale.
- **Bias Amplification:** Compressed models may inherit or amplify biases from training data.

- **Accessibility:** Making generative models more accessible could exacerbate existing challenges around content authenticity.

Responsible development requires careful consideration of these trade-offs and implementation of appropriate safeguards.

8.5 Concluding Remarks

This work demonstrates that substantial compression of diffusion models is achievable through strategic application of knowledge distillation, pruning, and architectural innovations. While our experimental results are limited by computational constraints, the theoretical frameworks and methodologies developed here provide a foundation for future research.

The integration of symmetry-aware architectures represents a particularly promising direction, offering a principled approach to improving both parameter efficiency and physical consistency. As diffusion models continue to advance toward practical deployment, combining these compression techniques with domain-specific inductive biases will be essential for realizing their full potential across diverse application domains.

The path toward truly compact, efficient, and reliable generative models remains challenging, but the techniques and insights presented in this work contribute meaningful steps toward that goal.

Bibliography

Appendix A

Additional Experimental Details

A.1 Hyperparameter Settings

Hyperparameter	Value
Batch size	64
Gradient accumulation steps	2
Learning rate	10^{-5}
Optimizer	AdamW
β_1	0.9
β_2	0.999
Weight decay	10^{-4}
EMA decay	0.995
Diffusion timesteps T	1000
$\beta_{\min}$	10^{-4}
$\beta_{\max}$	0.02
Mixed precision	FP16
Loss scale	Dynamic

Table A.1: Complete hyperparameter settings for all experiments.

A.2 Computational Requirements

Model	Training Time (hrs)	GPU Memory (GB)	FLOPs/sample
Teacher	48	14.2	2.1×10^{12}
Student	36	8.7	5.4×10^{11}
Pruned Student	42 (inc. fine-tune)	7.1	4.2×10^{11}

Table A.2: Computational requirements for model training and inference.